

← Installationsanleitungen

BANDspirit — Deployment-Optionen (V.2.0)

- 📘 **Aktuelle Architektur (V.2.0):** BANDspirit besteht aus zwei Container-Images — einem **Next.js 14 Frontend** und einem **C# ASP.NET Core 8 Backend** (REST + OData, Entity Framework Core). Dazu kommen PostgreSQL, Redis und ein S3-kompatibler Objektspeicher (MinIO). Das Deployment erfolgt containerbasiert — sowohl auf einem eigenen Ubuntu-Server (Variante 1) als auch in Azure (Variante 2). Die frühere Node.js-/Prisma-Deployment-Variante ist nicht mehr gültig.

📁 Variante 1 — Docker Container (Ubuntu Server)

Deployment des BANDspirit-Container-Stacks (Frontend, API, PostgreSQL, Redis, MinIO, Nginx) auf einem eigenen Ubuntu-Server via `docker compose`. Die vollständige Schritt-für-Schritt-Anleitung inkl. SSL/TLS (Let's Encrypt), Firewall, Monitoring, Backup und Sicherheitshärtung steht als Word-Dokument bereit.

Enthaltene Themen

- Server-Grundkonfiguration (Updates, Benutzer, SSH)
- Docker & Docker Compose Installation
- Repository klonen & `.env` konfigurieren
- Container-Stack starten (`docker compose up -d`)
- Nginx Reverse Proxy (Frontend + API)
- SSL/TLS mit Let's Encrypt (Certbot)
- Firewall-Konfiguration (UFW)
- Backup-Strategie (DB + Volumes)
- Update- & Rollback-Verfahren
- Sicherheitshärtung

Voraussetzungen

- Ubuntu Server 22.04 / 24.04 LTS
- Min. 2 vCPU, 4 GB RAM, 40 GB SSD
- Öffentliche IP + DNS-Eintrag
- SSH-Zugang (Root oder sudo)
- Docker Engine + Compose Plugin

📄 Anleitung herunterladen (Word)

📄 PDF

V.2.0 — Container-Stack (docker compose)

☁️ Variante 2 — Microsoft Azure Cloud (Container)

Deployment der beiden Container-Images (Frontend + API) in Azure — empfohlen über **Azure Container Apps** oder **App Service for Containers**, mit Azure Database for PostgreSQL, Azure Cache for Redis und Azure Blob Storage. Die Container-Images

werden in einer Azure Container Registry (ACR) verwaltet. Nachfolgend die Online-Kurzanleitung.

Inhaltsverzeichnis — Azure Cloud

- | | |
|--|--|
| 1. Voraussetzungen | 2. Zielarchitektur Azure (Container) |
| 3. Azure-Infrastruktur aufsetzen | 4. Source Code → Azure DevOps Repository |
| 5. CI/CD Pipeline (Container Build & Deploy) | 6. Azure Entra ID — SSO-Anbindung |
| 7. Umgebungsvariablen | 8. Datenbank-Migration (EF Core) |
| 9. Objektspeicher (Datei-Upload) | 10. Monitoring & Logging |
| 11. Go-Live Checkliste | |

1. Voraussetzungen

Folgende Voraussetzungen müssen vor Projektbeginn erfüllt sein:

Azure-Abonnement & Berechtigungen

- Aktives Azure-Abonnement (Pay-as-you-go, Enterprise Agreement oder CSP)
- Rolle `Contributor` oder höher auf Subscription-Ebene
- Zugriff auf **Azure Entra ID** (ehem. Azure AD) mit Berechtigung zur App-Registrierung
- Azure DevOps Organisation mit Projektberechtigung

Lokale Werkzeuge

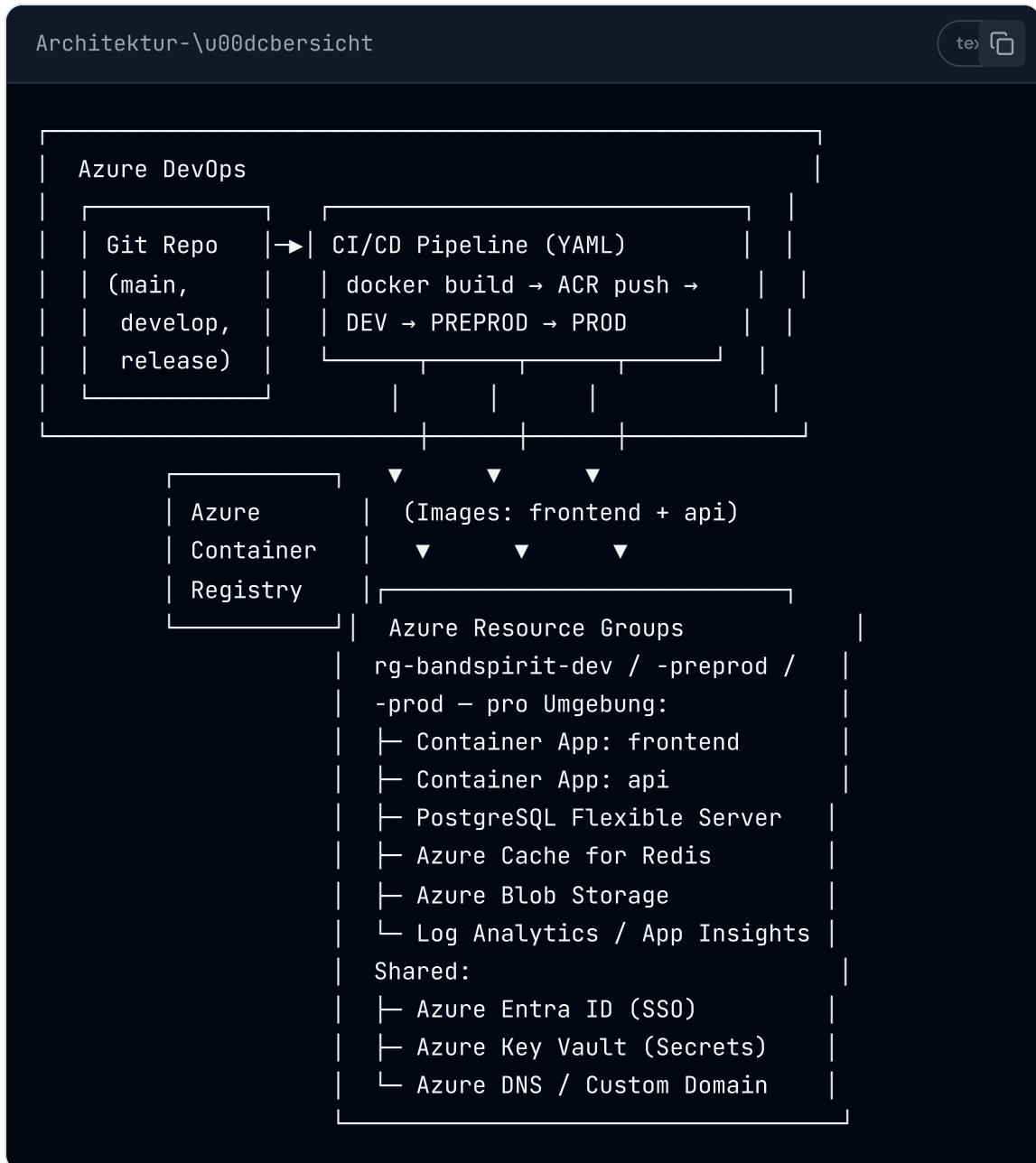
- `az` \u2014 Azure CLI \u2265 2.50
- `docker` \u2265 24 (Buildx für Multi-Image-Builds)
- `git` \u2265 2.30
- `psql` (PostgreSQL CLI) für DB-Migration

Applikations-Stack (V.2.0)

- **Frontend:** Next.js 14 (App Router), React 18, NextAuth.js 4 — als Docker-Image
- **Backend:** C# ASP.NET Core 8 (REST + OData), Entity Framework Core — als Docker-Image
- **Datenbank:** PostgreSQL 16
- **Cache / Rate-Limiting:** Redis 7
- **Objektspeicher:** S3-kompatibel (MinIO lokal, Azure Blob Storage in Azure)

2. Zielarchitektur Azure (Container)

Die folgende Architektur bildet drei isolierte Umgebungen (DEV / PREPROD / PROD) mit zwei Container-Images ab:



Frontend Container

Next.js 14 Standalone-Image (Port 3000).
Kommuniziert serverseitig mit dem API-Container.

API Container

ASP.NET Core 8 (Port 8080). REST (/api/*) +
OData (/odata), EF Core, JWT.

Azure PostgreSQL Flexible

Burstable B1ms (DEV) / General Purpose D2s
(PROD). SSL erzwungen, VNet für PROD.

Azure Cache for Redis

Für Rate-Limiting und Caching. Basic CO
(DEV) / Standard (PROD).

Azure Blob Storage

Azure Entra ID

S3-kompatibler Objektspeicher für Datei-Uploads (Logos, Dokumente).

SSO via OpenID Connect. Callback wird vom API-Backend verarbeitet (/api/auth/entra-callback).

> 3. Azure-Infrastruktur aufsetzen

3.1 Resource Groups & Container Registry

Azure CLI \u2014 Resource Groups + ACR

bas 

```
# Login
az login

# Variablen
LOCATION="switzerlandnorth" # Standort Schweiz
PREFIX="bandspirit"

# Resource Groups pro Umgebung
az group create --name rg-$PREFIX-dev      --location $LOCATION
az group create --name rg-$PREFIX-preprod  --location $LOCATION
az group create --name rg-$PREFIX-prod     --location $LOCATION

# Gemeinsame Container Registry (ACR)
az group create --name rg-$PREFIX-shared --location $LOCATION
az acr create --resource-group rg-$PREFIX-shared \
  --name acr${PREFIX} --sku Standard --admin-enabled false
```

3.2 Azure Database for PostgreSQL (Flexible Server)

PostgreSQL Flexible Server \u2014 pro Umgebung

bas 

```
# Beispiel für DEV (analog für preprod/prod mit angepasster SKU)
ENV="dev"
az postgres flexible-server create \
  --resource-group rg-$PREFIX-$ENV \
  --name pg-$PREFIX-$ENV \
  --location $LOCATION \
  --admin-user bandspiritadmin \
  --admin-password '<SICHERES_PASSWORT>' \
  --sku-name Standard_B1ms \
  --tier Burstable \
  --storage-size 32 \
  --version 16 \
```

```
--yes
```

```
# Datenbank anlegen
az postgres flexible-server db create \
  --resource-group rg-$PREFIX-$ENV \
  --server-name pg-$PREFIX-$ENV \
  --database-name bandspirit
```

⚠️ PROD-Empfehlung: Für Produktion mindestens Standard_D2s_v3 (General Purpose) verwenden und VNet-Integration aktivieren. SSL auf Require setzen.

3.3 Azure Cache for Redis

Redis Cache

bas 

```
az redis create \
  --resource-group rg-$PREFIX-$ENV \
  --name redis-$PREFIX-$ENV \
  --location $LOCATION \
  --sku Basic --vm-size c0
```

3.4 Container-Images bauen & nach ACR pushen

docker build + az acr

bas 

```
# ACR-Login
az acr login --name acr${PREFIX}
REGISTRY="acr${PREFIX}.azurecr.io"
TAG="1.0.0"

# Backend-Image (C# ASP.NET Core 8)
docker build -t $REGISTRY/bandspirit-api:$TAG ./backend
docker push $REGISTRY/bandspirit-api:$TAG

# Frontend-Image (Next.js 14)
docker build -t $REGISTRY/bandspirit-frontend:$TAG ./frontend
docker push $REGISTRY/bandspirit-frontend:$TAG
```

3.5 Azure Container Apps Environment

Container Apps \u2014 Environment + Apps

bas 

```
# Container Apps Environment (pro Umgebung)
az containerapp env create \
  --resource-group rg-$PREFIX-$ENV \
  --name cae-$PREFIX-$ENV \
  --location $LOCATION

# API-Container App (intern erreichbar)
az containerapp create \
  --resource-group rg-$PREFIX-$ENV \
  --name ca-$PREFIX-api-$ENV \
  --environment cae-$PREFIX-$ENV \
  --image $REGISTRY/bandspirit-api:$TAG \
  --target-port 8080 --ingress internal \
  --registry-server $REGISTRY

# Frontend-Container App (öffentlich erreichbar)
az containerapp create \
  --resource-group rg-$PREFIX-$ENV \
  --name ca-$PREFIX-frontend-$ENV \
  --environment cae-$PREFIX-$ENV \
  --image $REGISTRY/bandspirit-frontend:$TAG \
  --target-port 3000 --ingress external \
  --registry-server $REGISTRY
```

3.6 Azure Blob Storage

Storage Account + Container

bas 

```
az storage account create \
  --resource-group rg-$PREFIX-$ENV \
  --name st${PREFIX}${ENV} \
  --sku Standard_LRS \
  --kind StorageV2 \
  --location $LOCATION

az storage container create \
  --account-name st${PREFIX}${ENV} \
  --name uploads \
  --public-access off
```

3.7 Azure Key Vault (Secrets)

Key Vault f\u00fcr sichere Konfiguration

bas 

```

az keyvault create \
  --resource-group rg-$PREFIX-$ENV \
  --name kv-$PREFIX-$ENV \
  --location $LOCATION

# Secrets hinterlegen
az keyvault secret set --vault-name kv-$PREFIX-$ENV \
  --name ConnectionStrings--DefaultConnection \
  --value "Host=pg-$PREFIX-$ENV.postgres.database.azure.com;Port=5432;D

az keyvault secret set --vault-name kv-$PREFIX-$ENV \
  --name Jwt--Secret \
  --value "$(openssl rand -base64 48)"

az keyvault secret set --vault-name kv-$PREFIX-$ENV \
  --name NEXTAUTH-SECRET \
  --value "$(openssl rand -base64 32)"

```

3.8 Log Analytics / Application Insights

Monitoring

bas 

```

az monitor app-insights component create \
  --resource-group rg-$PREFIX-$ENV \
  --app ai-$PREFIX-$ENV \
  --location $LOCATION \
  --kind web

```

4. Source Code \u2192 Azure DevOps Repository

4.1 Azure DevOps Projekt erstellen

1. Navigiere zu <https://dev.azure.com/\u007borganisation\u007d>
2. Neues Projekt erstellen: **BANDspirit-KMS** (Visibility: Private)
3. Unter *Repos* wird automatisch ein leeres Git-Repository angelegt

4.2 Branching-Strategie

Branch	Umgebung	Trigger
develop	DEV	Push / Merge

Branch	Umgebung	Trigger
release/*	PREPROD	Push + manuelle Freigabe
main	PROD	Merge + manuelle Freigabe (4-Augen)

4.3 Repository-Struktur

```
Monorepo-Layout
te>

bandspirit/
├─ backend/           # C# ASP.NET Core 8 (Dockerfile)
├─ frontend/         # Next.js 14 (Dockerfile)
├─ docker/           # nginx-Konfiguration
├─ docker-compose.yml # lokaler / Ubuntu-Stack
└─ azure-pipelines.yml # CI/CD
```

4.4 Repository initialisieren

```
Git-Setup
bas>

cd /pfad/zum/bandspirit-quellcode

# .gitignore prüfen (node_modules/, .next/, bin/, obj/, .env)

git init
git checkout -b main
git remote add origin https://dev.azure.com/{org}/{project}/_git/BANDspirit
git add -A
git commit -m "Initial: BANDspirit KMS V.2.0"
git push -u origin main

git checkout -b develop
git push -u origin develop
```

Wichtig: Die Datei `.env` darf **nicht** ins Repository. Alle Secrets werden über Azure Key Vault und Container-App-Secrets verwaltet.

5. CI/CD Pipeline (Container Build & Deploy)

Die Pipeline baut beide Docker-Images, pusht sie in die ACR und deployt in die Container Apps.

5.1 Pipeline-Datei

azure-pipelines.yml

yan 

```
# azure-pipelines.yml - BANDspirit KMS V.2.0
# =====
trigger:
  branches:
    include: [ main, develop, release/* ]

pool:
  vmImage: 'ubuntu-latest'

variables:
  registry: 'acrbandspirit.azurecr.io'
  tag: '$(Build.BuildId)'

stages:
# ---- BUILD & PUSH IMAGES ----
- stage: Build
  displayName: 'Build & Push Container-Images'
  jobs:
    - job: BuildImages
      steps:
        - task: Docker@2
          displayName: 'Backend-Image (API)'
          inputs:
            containerRegistry: 'ACR-ServiceConnection'
            repository: 'bandspirit-api'
            command: 'buildAndPush'
            Dockerfile: 'backend/Dockerfile'
            buildContext: 'backend'
            tags: |
              $(tag)
        - task: Docker@2
          displayName: 'Frontend-Image'
          inputs:
            containerRegistry: 'ACR-ServiceConnection'
            repository: 'bandspirit-frontend'
            command: 'buildAndPush'
            Dockerfile: 'frontend/Dockerfile'
            buildContext: 'frontend'
            tags: |
              $(tag)
```

```
# ---- DEPLOY DEV ----
- stage: DeployDev
  displayName: 'Deploy → DEV'
  dependsOn: Build
  condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))
  jobs:
    - deployment: DeployDevJob
      environment: 'bandspirit-dev'
      strategy:
        runOnce:
          deploy:
            steps:
              - task: AzureCLI@2
                inputs:
                  azureSubscription: 'Azure-ServiceConnection'
                  scriptType: bash
                  scriptLocation: inlineScript
                  inlineScript: |
                    az containerapp update -n ca-bandspirit-api-dev \
                      -g rg-bandspirit-dev \
                      --image $(registry)/bandspirit-api:$(tag)
                    az containerapp update -n ca-bandspirit-frontend-dev \
                      -g rg-bandspirit-dev \
                      --image $(registry)/bandspirit-frontend:$(tag)

# ---- DEPLOY PREPROD (analog, Branch release/*) ----
# ---- DEPLOY PROD (analog, Branch main, dependsOn: DeployPreprod) ----
```

5.2 Freigabe-Gates konfigurieren

1. Azure DevOps \u2192 *Pipelines* \u2192 *Environments*
2. Environment `bandspirit-preprod` \u2192 *Approvals and checks* \u2192 Approver hinzufügen
3. Environment `bandspirit-prod` \u2192 Mind. 2 Approver (4-Augen-Prinzip)

5.3 Service Connections

1. *Project Settings* \u2192 *Service Connections*
2. **Docker Registry** \u2192 Azure Container Registry \u2192 Name `ACR-ServiceConnection`
3. **Azure Resource Manager** (Service Principal) \u2192 Name `Azure-ServiceConnection`

5.4 EF-Core-Migration im Deploy

```
# EF-Core-Migrationen werden beim Start des API-Containers automatisch
# angewendet (Database.Migrate() in Program.cs). Alternativ als Job:
- task: AzureCLI@2
  displayName: 'EF Core Migration (manuell, optional)'
  inputs:
    azureSubscription: 'Azure-ServiceConnection'
    scriptType: bash
    scriptLocation: inlineScript
    inlineScript: |
      az containerapp exec -n ca-bandspirit-api-$ENV \
        -g rg-bandspirit-$ENV \
        --command "dotnet BandSpirit.Api.dll --migrate"
```

 **Achtung:** Für PROD-Migrationen vorab ein Datenbank-Backup erstellen. EF-Core-Migrationen sind versioniert (Migrations/-Ordner) und werden mit `dotnet ef database update` bzw. beim Container-Start deterministisch angewendet.

6. Azure Entra ID \u2014 SSO-Anbindung

Azure Entra ID (ehem. Azure AD) wird als Identity Provider über **OpenID Connect** angebunden. Der OIDC-Callback wird vom **ASP.NET Core Backend** verarbeitet (POST `/api/auth/entra-callback`), das nach erfolgreicher Prüfung ein eigenes JWT ausstellt. Das Next.js-Frontend startet den Anmeldevorgang und speichert das Token in der NextAuth-Session.

6.1 App Registration erstellen

1. Azure Portal \u2192 *Microsoft Entra ID* \u2192 *App registrations* \u2192 *New registration*
2. Name: **BANDspirit KMS**
3. Account types: **Accounts in this organizational directory only** (Single Tenant)
4. Redirect URIs (Web):

Redirect URIs (alle Umgebungen)

te) 

```
https://ca-bandspirit-frontend-dev.<region>.azurecontainerapps.io/api/auth/entra-callback
https://ca-bandspirit-frontend-preprod.<region>.azurecontainerapps.io/api/auth/entra-callback
```

```
https://bandspirit.ihredomain.ch/api/auth/callback/entra
```

5. *Certificates & secrets* \u2192 *New client secret* \u2192 Wert notieren und in Key Vault speichern

6.2 API-Berechtigungen

1. *API permissions* \u2192 *Add* \u2192 *Microsoft Graph* \u2192 *Delegated*
2. Hinzufügen: `openid`, `profile`, `email`, `User.Read`
3. *Grant admin consent* klicken

6.3 Rollen-Mapping

Optionaler Gruppen-Claim (*Token configuration* \u2192 *Add groups claim*) für die Zuordnung Entra-Gruppe \u2192 BANDspirit-Rolle:

Entra-Gruppe	BANDspirit-Rolle
BANDspirit-Admin	ADMIN
BANDspirit-LeadLink	TEAMLEITUNG (Lead Link)
BANDspirit-Fachperson	FALLMANAGER (Fachperson BI)
BANDspirit-Sachbearbeiter	EINSATZPLANER (Sachbearbeiter:in)

6.4 Backend-Konfiguration (ASP.NET Core)

appsettings / Umgebungsvariablen

```
{
  "EntraId": {
    "TenantId": "<TENANT-ID>",
    "ClientId": "<CLIENT-ID>",
    "ClientSecret": "<aus Key Vault>",
    "Authority": "https://login.microsoftonline.com/<TENANT-ID>/v2.0"
  }
}
```

Als Umgebungsvariablen im Container: `EntraId__TenantId`, `EntraId__ClientId`, `EntraId__ClientSecret`. Das Backend validiert das OIDC-Token, legt SSO-Benutzer bei Bedarf an (Auto-Provisioning ohne lokales Passwort) und stellt ein eigenes JWT (8h Gültigkeit) aus.

❗ Dual-Auth: Das bestehende Credential-Login (E-Mail + Passwort, BCrypt) bleibt parallel zu SSO bestehen. Benutzer können sich wahlweise mit Entra ID oder lokalem Passwort anmelden.

7. Umgebungsvariablen

Variablen werden als Container-App-Secrets/-Env gesetzt. Sensible Werte über Key Vault References.

7.1 API-Container (ASP.NET Core)

```
az containerapp \u2014 API Env

ENV="dev"
az containerapp update -n ca-bandspirit-api-$ENV -g rg-bandspirit-$ENV \
--set-env-vars \
  ConnectionStrings__DefaultConnection=secretref:db-connection \
  Jwt__Secret=secretref:jwt-secret \
  Jwt__ExpiresHours=8 \
  Jwt__Issuer=BANDspirit \
  Jwt__Audience=BANDspirit \
  ConnectionStrings__Redis="redis-bandspirit-$ENV.redis.cache.windows.net" \
  Sntp__Host="smtp.office365.com" Sntp__Port=587 \
  Sntp__User="<MAIL-USER>" Sntp__Password=secretref:smtp-pw \
  S3__Endpoint="https://stbandspirit$ENV.blob.core.windows.net" \
  S3__Bucket="uploads" \
  EntraId__TenantId="<TENANT>" EntraId__ClientId="<CLIENT>" \
  EntraId__ClientSecret=secretref:entra-secret \
  ASPNETCORE_ENVIRONMENT="Production"
```

7.2 Frontend-Container (Next.js)

```
az containerapp \u2014 Frontend Env

az containerapp update -n ca-bandspirit-frontend-$ENV -g rg-bandspirit-$ENV \
--set-env-vars \
  NEXTAUTH_SECRET=secretref:nextauth-secret \
  NEXTAUTH_URL="https://bandspirit.ihredomain.ch" \
  API_BASE_URL="https://ca-bandspirit-api-$ENV.internal.<region>.azurecontainerapps.io" \
  NODE_ENV="production"
```

7.3 Variablen-Referenz

Variable	Container	Beschreibung
ConnectionStrings__DefaultConnection	API	PostgreSQL Connection String (EF Core, SSL)
Jwt__Secret	API	JWT-Signaturschlüssel (min. 32 Zeichen)

Variable	Container	Beschreibung
Jwt__ExpiresHours	API	Token-Gültigkeit in Stunden (Standard: 8)
ConnectionStrings__Redis	API	Redis (Rate-Limiting / Cache)
Smtp__Host / User / Password	API	SMTP (MailKit) für E-Mail-Versand
S3__Endpoint / Bucket	API	S3-kompatibler Objektspeicher (Blob)
EntraId__*	API	Entra ID SSO (Tenant, Client, Secret)
NEXTAUTH_SECRET	Frontend	NextAuth-Session-Signatur (min. 32 Zeichen)
NEXTAUTH_URL	Frontend	Öffentliche URL der Umgebung
API_BASE_URL	Frontend	Interne URL des API-Containers

i Key Vault References: Container-App-Secrets können auf Key-Vault-Einträge verweisen. Die Managed Identity der Container App benötigt die Rolle Key Vault Secrets User.

8. Datenbank-Migration (EF Core)

8.1 Schema auf Azure PostgreSQL anwenden

```

EF Core Migration

# Verbindung zur Azure-DB
export ConnectionStrings__DefaultConnection="Host=pg-bandspirit-dev.postgres.database.azure.com;Port=5432;Database=bandspirit;User Id=bandspirit@pg-bandspirit-dev.postgres.database.azure.com;Password=bandspirit;Pooling=true;MinPoolSize=10;MaxPoolSize=100;Command Timeout=30;Trusted_Connection=true;TrustServerCertificate=true;"

# Migrationen anwenden (aus dem backend-Verzeichnis)
dotnet ef database update

# Neue Migration erzeugen (bei Schema-Änderungen)
dotnet ef migrations add <Name>

# Hinweis: Beim Container-Start werden ausstehende Migrationen
# automatisch angewendet (Database.Migrate() in Program.cs).
```

8.2 Seed-Daten

Stammdaten, RBAC-Rollen/Permissions und Default-Benutzer werden über den DataSeeder beim Anwendungsstart initialisiert (idempotent — bereits vorhandene Datensätze werden nicht überschrieben).

8.3 Bestehende Daten migrieren

pg_dump / pg_restore

bas 

```
# Export aus bestehender DB
pg_dump --format=custom --no-owner --no-privileges \
  -h <AKTUELLE_DB_HOST> -U <USER> -d <DB_NAME> \
  > bandspirit_backup.dump

# Import in Azure PostgreSQL
pg_restore --no-owner --no-privileges \
  -h pg-bandspirit-prod.postgres.database.azure.com \
  -U bandspiritadmin -d bandspirit bandspirit_backup.dump
```

 **Wichtig:** Vor jeder Migration ein vollständiges Backup erstellen. Bei einer Migration aus einer bestehenden Umgebung zuerst das Schema per `dotnet ef database update` herstellen, dann Daten importieren.

9. Objektspeicher (Datei-Upload)

Das Backend nutzt einen **S3-kompatiblen Objektspeicher** über den `S3StorageService`. Lokal / auf dem Ubuntu-Server läuft dafür ein MinIO-Container; in Azure kann Azure Blob Storage (S3-kompatibel bzw. über die Blob-API) verwendet werden. Es ist **keine Code-Änderung** nötig — nur die Konfiguration (`S3__Endpoint`, `S3__Bucket`, Zugangsschlüssel) wird angepasst.

S3/Blob-Konfiguration (API-Container)

bas 

```
S3__Endpoint="https://stbandspirit<env>.blob.core.windows.net"
S3__Bucket="uploads"
S3__AccessKey="<ACCOUNT-NAME>"
S3__SecretKey="<ACCOUNT-KEY oder SAS>"
S3__UseSsl="true"
```

 **Betroffene Funktionen:** Firmen-Logo-Upload (`/api/firma/logo`) und BI-Kompass-Upload (`/api/bi-kompass`) laufen über den `S3StorageService`. Presigned URLs werden serverseitig generiert.

10. Monitoring & Logging

10.1 Logging

Das Backend protokolliert strukturiert über **Serilog** (Konsole + Datei). In Azure Container Apps werden die Container-Logs automatisch an **Log Analytics** weitergeleitet; zusätzlich kann Application Insights eingebunden werden.

10.2 Health Check

Health-Endpoint



```
# ASP.NET Core Health Checks (falls aktiviert):  
GET https://<api-host>/health  
  
# Alternativ Liveness über Dashboard-/OData-Ping:  
GET https://<api-host>/api/dashboard (mit gültigem JWT)
```

10.3 Empfohlene Alerts

- HTTP 5xx > 10/min \u2192 E-Mail + Teams-Benachrichtigung
- CPU > 80% über 5 Min \u2192 Warning
- DB Connections > 80% \u2192 Warning
- Response Time P95 > 3s \u2192 Warning
- Container-Restart-Loop \u2192 kritischer Alert

11. Go-Live Checkliste

Infrastruktur

- ☐ Resource Groups (dev/preprod/prod) + shared erstellt
- ☐ Azure Container Registry (ACR) erstellt
- ☐ PostgreSQL Flexible Server (v16) pro Umgebung provisioniert
- ☐ Azure Cache for Redis pro Umgebung erstellt
- ☐ Container Apps Environment + Apps (api/frontend) erstellt
- ☐ Blob Storage Accounts + Container erstellt
- ☐ Key Vault mit allen Secrets befüllt
- ☐ Managed Identity: Container App → Key Vault / ACR Zugriff
- ☐ Log Analytics / Application Insights konfiguriert

Repository & Pipeline

- ☐ Azure DevOps Projekt erstellt
- ☐ Monorepo (backend/frontend) in Azure Repos gepusht
- ☐ Branching-Strategie eingerichtet

- ☐ azure-pipelines.yml validiert (Build beider Images)
- ☐ Service Connections (ACR + ARM) konfiguriert
- ☐ Environments mit Freigabe-Gates erstellt
- ☐ Erster Pipeline-Durchlauf erfolgreich (Images in ACR)

Authentifizierung (Entra ID)

- ☐ App Registration erstellt
- ☐ Client Secret in Key Vault gespeichert
- ☐ Redirect URLs für alle Umgebungen eingetragen
- ☐ API Permissions + Admin Consent erteilt
- ☐ Backend EntraId__* Variablen gesetzt
- ☐ SSO-Login + Credential-Login (BCrypt) getestet
- ☐ Gruppen-/Rollen-Mapping geprüft (optional)

Datenbank & Storage

- ☐ EF-Core-Migrationen auf Azure DB angewendet
- ☐ DataSeeder-Initialdaten geprüft (Rollen/Permissions)
- ☐ Bestehende Daten migriert (falls vorhanden)
- ☐ S3/Blob-Konfiguration gesetzt (S3__*)
- ☐ Upload/Download (Logo, BI-Kompass) getestet
- ☐ DB Backup-Strategie konfiguriert

Testing & Go-Live

- ☐ Alle CRUD-Operationen (REST + OData) in DEV getestet
- ☐ RBAC-Berechtigungen pro Rolle geprüft
- ☐ Rate-Limiting (auth/api) verifiziert
- ☐ PREPROD-Abnahme durch Fachbereich
- ☐ Custom Domain + SSL-Zertifikat (PROD)
- ☐ DNS-Einträge gesetzt
- ☐ PROD-Deployment via Pipeline
- ☐ Smoke-Test: Login, Dashboard, CRUD
- ☐ Monitoring-Alerts verifiziert
- ☐ Rollback-Plan dokumentiert